```
    for(unsigned i=0;i<buckets.size();++i)
    {
        locks.push_back(
            std::unique_lock<boost::shared_mutex>
(buckets[i].mutex));
    }
    std::map<Key,Value> res;
    for(unsigned i=0;i<buckets.size();++i)
    {
        for(bucket_iterator it=buckets[i].data.begin();
            it!=buckets[i].data.end();
            ++it)
        {
             res.insert(*it);
        }
    }
    return res;
}
```

The lookup table implementation from listing 6.11 increases the opportunity for concurrency of the lookup table as a whole by locking each bucket separately and by using a `boost::shared_mutex` to allow reader concurrency on each bucket. But what if you could increase the potential for concurrency on a bucket by even finer-grained locking? In the next section, you'll do just that by using a thread-safe list container with iterator support.

6.3.2. Writing a thread-safe list using locks

A list is one of the most basic data structures, so it should be straightforward to write a thread-safe one, shouldn't it? Well, that depends on what facilities you're after, and you need one that offers iterator support, something I shied away from adding to your map on the basis that it was too complicated. The basic issue with STL-style iterator support is that the iterator must hold some kind of reference into the internal data structure of the container. If the container can be modified from another thread, this reference must somehow remain valid, which essentially requires that the iterator hold a lock on some part of the structure. Given that the lifetime of an STL-style iterator is completely outside the control of the container, this is a bad idea.